

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

ОТЧЁТ
Лабораторная работа №3

Выполнила:

Красноперова Е. Д.
Группа: К3443

Проверил:

Добряков Д. И.

Санкт-Петербург,

2026 г.

Задача

Реализовать Dockerfile для каждого сервиса, написать общий docker-compose.yml и настроить сетевое взаимодействие между сервисами.

Ход работы

В проекте уже реализованы API Gateway, UserService, RecipeService и SocialService, отдельные базы данных сервисов, синхронное HTTP-взаимодействие и асинхронный обмен событиями через RabbitMQ.

В ДЗ4 были определены границы сервисов: UserService отвечает за пользователей и JWT-аутентификацию, RecipeService - за рецепты, ингредиенты и шаги, SocialService - за лайки, комментарии, сохранения и подписки, API Gateway является единой точкой входа. В ЛР2 это разделение было реализовано программно. В ДЗ5 в проект был добавлен RabbitMQ с событиями recipe.social.updated и recipe.deleted.

1. Исходная архитектура проекта

В контейнеризируемый проект входят четыре уже реализованных приложения:

- API Gateway — порт 3000, единая точка входа клиента.
- UserService — порт 3001, пользователи, регистрация, JWT и профиль.
- RecipeService — порт 3002, рецепты, шаги, ингредиенты и агрегированная социальная статистика.
- SocialService — порт 3003, лайки, комментарии, сохранения и подписки.

Согласно архитектуре database-per-service используются три отдельные PostgreSQL базы: recipes_users, recipes_recipes и recipes_social. RabbitMQ используется для событий из ДЗ5.

2. Dockerfile для сервисов

Для каждого существующего сервиса добавлен отдельный Dockerfile. Используется многоэтапная сборка: на первом этапе устанавливаются зависимости и выполняется TypeScript-компиляция, на втором запускается собранный JavaScript-код с production-зависимостями.

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
COPY services/api-gateway/package.json services/api-gateway/package.json
```

```

COPY services/user-service/package.json services/user-service/package.json
COPY services/recipe-service/package.json services/recipe-service/package.json
COPY services/social-service/package.json services/social-service/package.json
RUN npm ci
COPY services/recipe-service services/recipe-service
RUN npm run build -w recipe-service

FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
COPY services/api-gateway/package.json services/api-gateway/package.json
COPY services/user-service/package.json services/user-service/package.json
COPY services/recipe-service/package.json services/recipe-service/package.json
COPY services/social-service/package.json services/social-service/package.json
RUN npm ci --omit=dev
COPY --from=builder /app/services/recipe-service/dist services/recipe-service/dist
CMD ["node", "services/recipe-service/dist/index.js"]

```

Аналогичные Dockerfile находятся в каталогах api-gateway, user-service и social-service. Исходный код этих сервисов при выполнении ЛРЗ не заменяется.

3. Общий docker-compose.yml

Вместо отдельного контейнера только для RabbitMQ, который использовался в ДЗ5, в ЛРЗ общий docker-compose.yml описывает всю систему. Он включает четыре приложения, три PostgreSQL и RabbitMQ. Для каждой базы создан отдельный именованный volume.

Для PostgreSQL заданы healthcheck. Приложения запускаются после готовности соответствующей базы. RabbitMQ также имеет healthcheck.

4. Сетевое взаимодействие между сервисами

Все контейнеры подключены к общей bridge-сети recipes-net. Docker Compose предоставляет встроенное DNS-разрешение имён сервисов. Поэтому внутри контейнеров используются адреса вида http://user-service:3001 и http://recipe-service:3002.

```

api-gateway -> user-service:3001
api-gateway -> recipe-service:3002
api-gateway -> social-service:3003

user-service -> user-db:5432
recipe-service -> recipe-db:5432
social-service -> social-db:5432

recipe-service -> rabbitmq:5672
social-service -> rabbitmq:5672

```

Это принципиально отличается от локального запуска, где использовался localhost. В Docker localhost внутри контейнера означает сам контейнер, поэтому для связи между контейнерами применяются имена сервисов.

5. RabbitMQ из ДЗ5 в контейнерной среде

Существующая логика ДЗ5 сохраняется. RecipeService использует очередь recipe-service.social-updates и подписывается на события recipe.social.updated и recipe.deleted. SocialService использует очередь social-service.recipe-events и подписывается на recipe.deleted.

Внутри Docker оба сервиса подключаются к RabbitMQ по адресу amqp://guest:guest@rabbitmq:5672. На хосте management-интерфейс опубликован на порту 15673, а AMQP — на 5673, чтобы не конфликтовать с ранее использовавшимся локальным RabbitMQ на 15672/5672.

6. Базы данных

Для сохранения принципа database-per-service созданы три независимых контейнера PostgreSQL:

Сервис	Контейнер БД	База
UserService	user-db	recipes_users
RecipeService	recipe-db	recipes_recipes
SocialService	social-db	recipes_social

Сервисы обращаются только к своей базе. Межсервисные данные по-прежнему передаются через HTTP API или RabbitMQ, как было реализовано в предыдущих работах.

7. Запуск проекта

Запуск всей системы выполняется одной командой из корня проекта:

```
docker compose up -d --build
docker compose ps
```

Просмотр логов:

```
docker compose logs -f api-gateway
docker compose logs -f recipe-service
docker compose logs -f social-service
```

Остановка:

```
docker compose down
```

Остановка с удалением данных:

```
docker compose down -v
```

8. Проверка

После запуска необходимо убедиться, что контейнеры API Gateway, UserService, RecipeService, SocialService, три PostgreSQL и RabbitMQ находятся в состоянии running/Up.

Функциональная проверка выполняется через существующий Postman-сценарий проекта: регистрация пользователя, создание рецепта, социальная операция и обращение к API через Gateway. Для RabbitMQ дополнительно проверяются запуск consumers и результаты обработки событий, которые ранее были подтверждены в ДЗ5.

Вывод

Для каждого микросервиса создан Dockerfile, а общий docker-compose.yml объединяет API Gateway, UserService, RecipeService, SocialService, три независимые PostgreSQL базы и RabbitMQ. Межсервисное взаимодействие организовано через общую Docker-сеть и DNS-имена сервисов. Функциональная архитектура и механизмы межсервисного взаимодействия предыдущих работ сохраняются.